



Module M244 • Veille Technologique

RADAR

Realise par

KONATE BACHIROU
SYLLA KARAMO

Encadre par

Pr. Younes WADIAI

Année universitaire 2025 – 2026

Remerciements

Nous adressons nos remerciements les plus sinceres au Pr. Younes Wadiat, enseignant du module M244 Veille Technologique a l'Ecole Nationale des Sciences Appliquees de Tetouan, pour la rigueur de son enseignement et la clarte de la methodologie qu'il nous a transmise. Sans le cadre conceptuel du cycle de veille, des grilles d'analyse SWOT et PESTEL, de la grille CRAAP et des signaux faibles que nous avons recus en cours, ce projet n'aurait pu prendre la forme d'un produit operationnel. Nous lui sommes egalement reconnaissants pour la liberte qu'il nous a laissee dans le choix d'un sujet ambitieux, et pour son ouverture d'esprit face a une realisation qui croise sa specialite avec l'intelligence artificielle generative.

Resume

Ce rapport presente RADAR, un outil pense pour les entreprises qui veulent savoir au quotidien ce que font leurs concurrents sans y passer plusieurs heures chaque matin.

L'utilisation est directe. On indique le nom de son entreprise et celui de ses concurrents principaux, et RADAR s'occupe du reste. Chaque matin, il parcourt le web pour reperer les nouvelles informations qui touchent ces concurrents : changements de prix, recrutements importants, lancements de produits, levees de fonds, campagnes de communication. L'intelligence artificielle qui pilote l'outil lit ces informations, fait le tri entre celles qui sont fiables et celles qui ne le sont pas, et ne retient que ce qui merite vraiment l'attention.

Vers sept heures du matin, un compte-rendu structure arrive dans la boite mail de l'utilisateur. On y trouve un resume rapide des mouvements de la veille, une analyse des forces et faiblesses de l'entreprise face a la concurrence, une lecture des grandes tendances de son secteur, une liste d'indices qui annoncent des evolutions a venir, et des recommandations classees par ordre d'importance. Le meme contenu reste accessible dans un tableau de bord en ligne, avec un export en PDF pour les reunions ou les livrables clients.

Table des matières

Remerciements	1
Resume	2
Liste des acronymes et abreviations	7
1 Introduction et vue d'ensemble	8
1.1 Introduction generale	8
1.2 Problematique et besoin identifie	8
1.3 Objectifs et solution	9
1.4 Methodologie projet et planification	10
1.5 Cahier des charges simplifié	12
2 Etat de l'art	13
2.1 Concepts fondateurs de la veille strategique	13
2.2 Panorama des outils de veille existants	13
2.3 Evolution des agents IA autonomes	14
2.4 Positionnement de RADAR	15
3 Architecture de RADAR	16
3.1 Schema global du systeme	16
3.2 Choix d'architecture cles	17
3.3 Stack technique justifiee	18
3.4 Securite et isolation des sessions utilisateur	20
4 Implementation du moteur agent	21
4.1 Infrastructure Docker	21
4.2 Configuration OpenClaw	21
4.3 Architecture skills versus agents	23
4.4 Les huit competences SKILL.md	24
4.4.1 Deep Research	24
4.4.2 Orchestrateur	24
4.4.3 Collecteur	25
4.4.4 Evalueur	26
4.4.5 Analyste PESTEL	26
4.4.6 Analyste SWOT	27
4.4.7 Detecteur de signaux faibles	27

4.4.8	Redacteur	28
4.5	Integration de DeepSeek V4 Pro	28
4.6	Integration de Tavily et fallback DuckDuckGo	29
4.7	Communication entre agent et application web	29
4.8	Gestion des timeouts et fiabilite du pipeline	31
5	Implementation de l'application web	32
5.1	Stack Next.js 16, React 19 et Tailwind CSS	32
5.2	Authentification et gestion utilisateur	32
5.3	Onboarding et tableau de bord	32
5.4	Routes API internes	32
5.5	Modele de donnees Prisma et PostgreSQL	32
5.6	Generation PDF et envoi par courrier electronique	32
6	Tests et validation du systeme	33
6.1	Tests unitaires	33
6.1.1	Cas Marjane : test isole de Deep Research	33
6.1.2	Cycle complet : rapport-test-005	33
6.1.3	Echecs rencontres et corrections	34
6.2	Tests d'integration	35
7	Bilan et perspectives	36
7.1	Analyse critique	36
7.1.1	Forces du projet	36
7.1.2	Faiblesses et limites de la V1	37
7.1.3	Difficultes rencontrees et solutions trouvees	37
7.2	Apports du projet	38
7.3	Conclusion generale	39
7.4	Perspectives d'evolution	39
8	Bibliographie et references	41

Table des figures

1.1	Diagramme de Gantt de la planification du projet RADAR	11
3.1	Architecture globale de RADAR	16
4.1	Configuration retenue : un agent chargeant huit competences	23

Liste des tableaux

- 1.1 Outils de travail et de coordination utilises sur le projet RADAR 11
- 1.2 Cahier des charges synthetique de RADAR V1 12
- 2.1 Panorama compare des principaux outils de veille concurrentielle 14
- 3.1 Stack technique de RADAR et justification de chaque brique 19
- 4.1 Comparatif des modeles LLM evalues pour RADAR 28
- 4.2 Routes internes consommees par le moteur agent 30
- 6.1 Resultats du cycle de test rapport-test-005 (mai 2026) 34
- 7.1 Principales difficultes techniques et solutions apportees 38

Liste des acronymes et abreviations

API	Application Programming Interface
BDIA	Big Data et Intelligence Artificielle
CRAAP	Currency, Relevance, Authority, Accuracy, Purpose
CRUD	Create, Read, Update, Delete
ENSA	Ecole Nationale des Sciences Appliquees
ERD	Entity Relationship Diagram
HTTP	Hypertext Transfer Protocol
ICP	Ideal Customer Profile
JSON	JavaScript Object Notation
LLM	Large Language Model
M244	Code du module Veille Technologique
MVP	Minimum Viable Product
ORM	Object Relational Mapping
OAuth	Open Authorization
PDF	Portable Document Format
PESTEL	Politique, Economique, Sociologique, Technologique, Environnemental, Legal
PME	Petite et Moyenne Entreprise
PRD	Product Requirements Document
RGPD	Reglement General sur la Protection des Donnees
SaaS	Software as a Service
SSO	Single Sign-On
SWOT	Strengths, Weaknesses, Opportunities, Threats
TPM	Tokens Per Minute
UI	User Interface

Chapitre 1

Introduction et vue d'ensemble

1.1 Introduction generale

Ce rapport restitue le travail consacres a la conception et au developpement du projet RADAR, realise dans le cadre du module M244 Veille Technologique du cycle ingenieur Big Data et Intelligence Artificielle de l'Ecole Nationale des Sciences Appliquees de Tetouan.

Le module M244, vise a former les futurs ingenieurs aux pratiques professionnelles de la veille strategique : identification des besoins informationnels, collecte methodique, evaluation des sources, analyse a travers les grilles SWOT et PESTEL, detection des signaux faibles, et diffusion structuree des resultats. La validation du module exige un projet de fin de semestre qui demontre la maitrise concrete de ces concepts.

Au lieu de réaliser le processus de veille pour une entreprises en particulier, nous avons fait le pari de livrer un produit logiciel : une plateforme qui automatise le cycle de veille de bout en bout et ça pour n'importe quel intreprise, en s'appuyant sur les capacites recentes des grands modeles de langage. Ce choix repond a une triple intention. D'abord traduire chaque concept appris en cours en une brique technique testable. Ensuite obtenir un outil reellement deployable, utilisable par des dirigeants de PME ou des consultants sans formation prealable. Enfin construire une realisation defendable lors des premiers entretiens d'embauche en strategie, en data ou en IA appliquee.

1.2 Problematique et besoin identifie

Toute entreprise qui evolue dans un environnement concurrentiel doit savoir ce que font ses rivaux. Les dirigeants de PME, les responsables strategie, les consultants en mission le savent : un concurrent qui ajuste ses prix, recrute massivement, leve des fonds ou refond son offre constitue une information qui peut basculer une decision strategique. Le probleme n'est pas que ces informations soient indisponibles, elles sont au contraire largement publiques. Le probleme est que personne, dans une organisation de taille moyenne, n'a le temps de les collecter quotidiennement, de les recouper, de les evaluer puis de les synthetiser sous une forme exploitable.

En pratique, la veille concurrentielle prend rarement une forme aboutie dans les

PME et même dans les grandes entreprises. La situation la plus fréquente : aucune routine formalisée, la direction se contente du bouche-a-oreille sectoriel et de ce qui remonte sur LinkedIn. Variante un peu plus mature : un collaborateur dédié une heure ou deux chaque matin à naviguer sur les sites concurrents, scroller les réseaux, lire la presse économique, et consigne ce qu'il en retient dans un tableur sans grille d'analyse. Cas plus rare : l'entreprise s'abonne à Similarweb, SEMrush ou Crayon, mais finit par constater que ces outils répondent surtout à des besoins SEO et marketing, pas à une veille stratégique au sens du cycle enseigné en M244.

Cinq défaillances reviennent régulièrement dans ce mode opératoire. La découverte tardive d'abord : un changement de pricing ou une levée de fonds remonte par la presse plusieurs semaines après les faits. L'absence de méthode d'évaluation des sources ensuite, qui aboutit à mélanger rumeurs et informations confirmées sans hiérarchie. L'absence de synthèse aux formats SWOT ou PESTEL, ce qui rend très difficile un raisonnement sur les axes stratégiques. L'absence de détection des signaux faibles, ces indices précoces et dispersés qui annoncent un mouvement de fond. L'absence enfin d'une mémoire centralisée : chaque revue concurrentielle reconstruit l'historique à partir de zéro.

Le besoin tient en une phrase : disposer chaque matin d'une synthèse concurrentielle prête à être lue, fondée sur la méthodologie M244, sans qu'un humain n'ait à passer son temps à la produire. Le dispositif doit couvrir le cycle complet, de la collecte à la diffusion, en passant par l'analyse, et ne remonter que des informations vérifiées et recoupées. C'est exactement le périmètre de RADAR.

1.3 Objectifs et solution

L'objectif principal est simple : livrer une plateforme qui exécute toute seule le cycle de veille, depuis la collecte des informations jusqu'à la livraison d'un rapport quotidien. Cinq objectifs opérationnels en découlent.

Automatiser d'abord la collecte web sans déclencher de blocage anti-bot, en passant par un fournisseur de recherche conçu pour les agents. Évaluer ensuite chaque source selon la grille CRAAP avant de l'intégrer dans un rapport, pour que la fiabilité des informations remontées soit traçable. Produire des analyses SWOT et PESTEL exploitables, dont la qualité résiste à un examen critique. Détecter les signaux faibles, c'est-à-dire les indices précoces qui annoncent une évolution sectorielle. Livrer enfin le tout dans un tableau de bord web sobre et dans un digest courrier électronique quotidien.

La solution proposée s'articule autour d'un agent d'intelligence artificielle orchestré par OpenClaw, un runtime open source qui exécute des compétences spécialisées décrites au format Markdown. L'agent RADAR mobilise huit compétences distinctes :

une competence Deep Research qui caracterise le profil de l'entreprise utilisatrice lors de l'inscription (etape independante du cycle quotidien), un orchestrateur qui coordonne le cycle de veille, un collecteur qui interroge Tavily pour rapporter des sources fraiches sur chaque concurrent, un evaluateur qui calcule le score CRAAP de chaque source, deux analystes qui produisent respectivement la grille PESTEL puis la grille SWOT, un detecteur de signaux faibles, et un redacteur qui synthetise l'ensemble en un rapport narratif. L'orchestrateur enchainne ces competences via le mecanisme natif de spawning de sessions d'OpenClaw, ce qui garantit l'isolation des contextes et une consommation maitrisee des tokens.

L'utilisateur, qu'il soit dirigeant de PME ou grand groupe, consultant ou etudiant en strategie, accede a RADAR par une application web batie sur Next.js 16. Il renseigne le nom de son entreprise et son site web a l'inscription, ajoute ses concurrents prioritaires et selectionne ses axes de surveillance. Le moteur agent prend ensuite le relais : chaque matin a six heures, un cron declenche l'orchestrateur qui execute la chaine complete, persiste les resultats en base via les routes internes de l'application web, et notifie l'utilisateur. La consultation s'effectue dans un tableau de bord qui presente le feed des alertes, les grilles SWOT et PESTEL, la timeline des mouvements detectes et les signaux faibles identifies. Un export PDF permet enfin de partager le rapport en comite de direction ou en livrable client.

1.4 Methodologie projet et planification

La conduite du projet s'est inspiree des principes du developpement iteratif et de la philosophie Lean Startup. Plutot que de specifier l'integralite du produit avant d'ecrire la premiere ligne de code, nous avons procede par cycles courts d'une a deux semaines. Chaque cycle se concluait par un livrable demonstrable, soit un composant technique mis bout a bout, soit une serie de tests d'integration. Cette approche nous a permis d'absorber sans douleur les decisions techniques qu'on ne pouvait pas anticiper depuis le debut, comme la migration du modele GPT-4o vers DeepSeek V4 Pro a cause des limites de tokens par minute, ou le passage du fournisseur de recherche DuckDuckGo a Tavily a cause de la detection des rafales de requetes.

Les outils mobilises pour la coordination et le developpement sont resumes dans le tableau 1.1.

Table 1.1. Outils de travail et de coordination utilises sur le projet RADAR

Categorie	Outils retenus
Versionnement et collaboration	Git, GitHub (branches feature/agent et feature/web , pull requests)
Gestion du monorepo	pnpm 10, Turborepo
Documentation interne	Fichiers Markdown versionnes
Conteneurisation	Docker, Docker Compose, image officielle <code>ghcr.io/openclaw/openclaw:latest</code>
Tests d'integration	Mock API HTTP en Node.js qui simule les routes Next.js cote agent
Editeur de code	Visual Studio Code, LaTeX (pour le rapport), ESLint, Prisma
Communication equipe	WhatsApp pour le quotidien, sessions de travail synchrones le week-end

La planification du projet s'est etalee entre mars et juin 2026, en suivant l'enchainement de phases presente dans la figure 1.1.

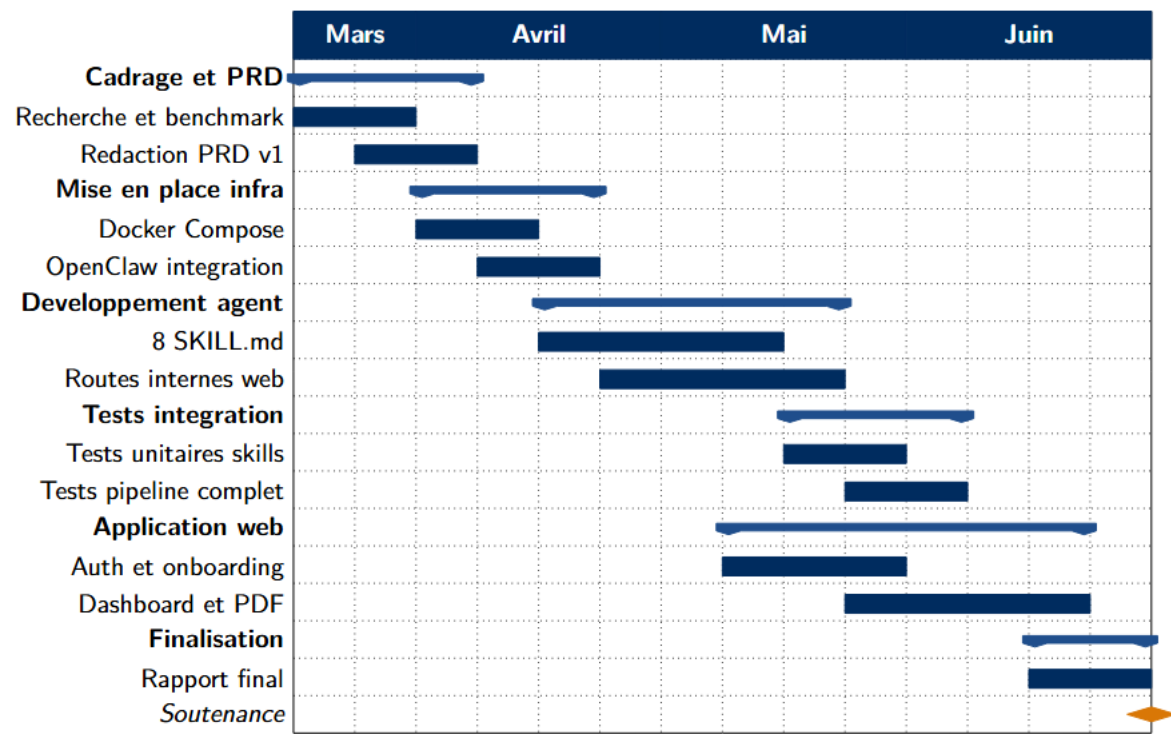


Figure 1.1. Planification du projet RADAR sur quatre mois (mars a juin 2026)

Figure 1.1. Diagramme de Gantt de la planification du projet RADAR

1.5 Cahier des charges simplifié

RADAR n'est pas qu'un exercice scolaire : nous l'avons conçu comme une offre que l'on pourrait facturer à un client. Le tableau 1.2 formalise le cahier des charges de la version 1 dans cette logique de prestation.

Table 1.2. Cahier des charges synthétique de RADAR V1

Rubrique	Engagement
Besoin client	Disposer chaque matin d'un rapport de veille concurrentielle structure, sans intervention manuelle, couvrant entre un et dix concurrents nommés par l'utilisateur.
Fonctionnalités clés	Inscription guidée avec enrichissement automatique du profil entreprise (Deep Research), cycle de veille quotidien déclenché par cron, dashboard temps réel, digest courrier électronique quotidien, export PDF.
Contraintes techniques	Hebergement Docker mutualisé ou self-hosted (VPS Hetzner CX21), dépendance externe à DeepSeek (modèle LLM) et Tavily (recherche web), réseau interne entre les services, persistance PostgreSQL.
Indicateurs de succès	Rapport produit chaque jour avant 7h00, sources évaluées CRAAP avec score moyen supérieur à 14 sur 20, recoupement minimum à deux sources indépendantes pour chaque alerte, taux d'échec inférieur à 5 pour cent par cycle.
Délai de livraison	Une version opérationnelle déployée sur VPS au plus tard une semaine avant la soutenance, c'est-à-dire mi-juin 2026.
Ressources mobilisées	Deux développeurs étudiants en cycle ingénieur, environ trente heures hebdomadaires chacun sur la durée du semestre.
Résultats attendus	Une plateforme utilisable en mode démonstration pour le jury, un rapport documenté, un dépôt Git public, et la capacité à soutenir une démonstration en direct sur un compte test.

Ce cahier des charges servira de référence tout au long du rapport : chaque choix technique présenté dans les chapitres suivants pourra être confronté aux engagements pris ici, et le chapitre 7 établira un bilan honnête de ce qui a été tenu et de ce qui reste à finaliser.

Chapitre 2

Etat de l'art

Ce chapitre situe RADAR par rapport a deux univers qui le precedent : la veille strategique en tant que discipline, et l'ecosysteme des agents IA en tant que technologie. Il reste volontairement bref, l'objectif est de marquer et de fournir une comprehension plus large et de differencier notre solution du marche actuel.

2.1 Concepts fondateurs de la veille strategique

La veille strategique, discipline formalisee dans la litterature francophone par Humbert Lesca, est enseignee dans le module M244 du Pr. Younes Wadii a travers quatre concepts que nous avons transposes directement dans le moteur RADAR.

Le **cycle de veille** en cinq etapes (identification des besoins, collecte, analyse et traitement, diffusion et exploitation, mise a jour continue) structure le pipeline de RADAR de bout en bout : chaque competence du moteur correspond a un segment du cycle.

L'evaluation des sources s'appuie sur la **grille CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose), formalisee par Sarah Blakeslee en 2004. Chaque critere est note sur quatre points pour un score global sur vingt. La competence Evalueur calcule ce score pour chaque source ; un seuil de cinq sur vingt declenche le filtrage.

L'analyse strategique mobilise les grilles **SWOT** (forces et faiblesses internes, opportunités et menaces externes) et **PESTEL** (six dimensions : politique, economique, sociologique, technologique, environnementale, legale). RADAR produit la SWOT quotidiennement et la PESTEL au rythme hebdomadaire.

Enfin, RADAR detecte les **signaux faibles** theorises par Igor Ansoff en 1975 dans *Managing Strategic Surprise by Response to Weak Signals*. La competence dediee croise les sources mineures (forums, blogs, offres d'emploi) et majeures sur une fenetre glissante de trente jours stockee en base PostgreSQL, pour faire emerger des recurrences qui annoncent un mouvement de fond.

2.2 Panorama des outils de veille existants

Le marche de la veille concurrentielle est dense mais segmente. Les outils les plus connus repondent en realite a des besoins assez differents les uns des autres, et la

plupart ne couvrent pas la veille strategique methodologique au sens du M244. Le tableau 2.1 dresse un panorama des solutions que nous avons examinees lors de la phase de cadrage.

Table 2.1. Panorama compare des principaux outils de veille concurrentielle

Outil	Coeur metier	Limite par rapport a RADAR
Similarweb	Mesure du trafic web, classement par categorie, sources d'audience	Aucune analyse SWOT ni PESTEL, pas de detection de signaux faibles, oriente SEO et marketing
SEMrush	Positionnement de mots-cles, backlinks, suivi de la presence digitale	Meme limite : centre sur le SEO, ne couvre pas la veille strategique
Crayon	Veille concurrentielle entreprise avec battlecards et alerting	Tres oriente sales enablement grand compte, prix prohibitif pour une PME
Mention	Social listening, mentions de marque, sentiment	Pas de cadre methodologique, ne produit pas de grille d'analyse
Brandwatch	Social listening grand compte avec analytics avances	Meme remarque, plus la barriere du prix
Google Alerts	Notification de nouveautes a partir de mots-cles	Beaucoup trop bruyant, aucune evaluation des sources, pas de structuration
Feedly	Agregateur de flux RSS avec couche IA optionnelle	Centre sur la lecture, ne genere pas de synthese strategique

Aucun de ces outils ne fait du cycle M244 son fil conducteur. RADAR se positionne sur ce creneau libre : produire chaque jour des livrables strategiques structures par une methodologie explicite, a destination des PME et des consultants independants qui n'ont ni le budget ni le besoin des suites entreprise.

2.3 Evolution des agents IA autonomes

Le terme *agent* designe depuis longtemps en intelligence artificielle un programme capable de percevoir son environnement et d'agir pour atteindre un objectif. Le changement majeur intervient en 2023 avec l'arrivee des grands modeles de langage : le moteur de raisonnement n'est plus une suite de regles ecrites a la main, mais un LLM auquel on donne acces a des outils externes et a une memoire de travail. AutoGPT, publie en avril 2023, a popularise cette approche, suivi par des frameworks plus aboutis comme LangChain et son evolution LangGraph.

Les architectures actuelles reposent sur quatre briques : un LLM comme moteur de raisonnement, des outils pour agir au-delà du texte (recherche web, lecture de fichiers, execution de code), une memoire a court et long terme, et une boucle de controle qui decide quand mobiliser chaque outil.

OpenClaw, le runtime que nous avons retenu pour RADAR, integre ces briques dans un seul binaire compatible avec l'API OpenAI Chat Completions. Les comportements d'un agent se decrivent dans des fichiers Markdown `SKILL.md`, lisibles et modifiables sans recompilation. La fonction `sessions_spawn` cree des sous-sessions isolees indispensables a un pipeline sequentiel, le cron natif gere le declenchement quotidien, et les outils `web_search`, `web_fetch` et `browser` sont disponibles sans configuration. Face a un paysage mouvant (Claude Agent SDK, Mastra, LangGraph, plateformes hebergees), notre choix repose sur le caractere open source du code, la compatibilite native avec l'API OpenAI qui ouvre la porte a tous les modeles disponibles dans cette interface, et la lisibilite immediate du format `SKILL.md`.

2.4 Positionnement de RADAR

RADAR n'a pas vocation a remplacer Similarweb, ni a jouer le role d'un agregateur de flux, d'un social listener ou d'un framework d'agents generaliste. Sa specialite tient en une formule : automatiser le cycle de veille M244 pour des PME et des consultants, en transformant chaque concept du cours en une fonctionnalite produit observable.

Cette specialisation est a la fois la force et la limite du projet. Force, parce qu'elle assure une coherence methodologique que les outils generalistes n'ont pas : chaque source collectee est evaluee par CRAAP, chaque alerte est recoupee, chaque rapport suit la structure attendue par le cours. Limite, parce qu'elle restreint le perimetre fonctionnel et nous oblige a laisser de cote des sujets adjacents comme la veille brevet, la veille reglementaire ou la surveillance de la reputation en ligne. Ces sujets pourraient faire l'objet d'extensions, dont nous discutons dans les perspectives du chapitre 7.

Chapitre 3

Architecture de RADAR

Ce chapitre presente l'architecture technique de RADAR : la maniere dont les composants s'articulent, les decisions structurantes qui ont oriente la conception, la stack retenue et les precautions prises pour l'isolation des sessions utilisateur.

3.1 Schema global du systeme

RADAR fonctionne sur trois services Docker independants qui collaborent via un reseau interne. Le premier est l'application web batie sur Next.js qui sert d'interface utilisateur et de couche de persistance. Le deuxieme est le moteur agent OpenClaw qui execute les huit competences specialisees. Le troisieme est la base de donnees PostgreSQL qui stocke profils, sources, analyses et rapports. La figure 3.1 schematise cette topologie.

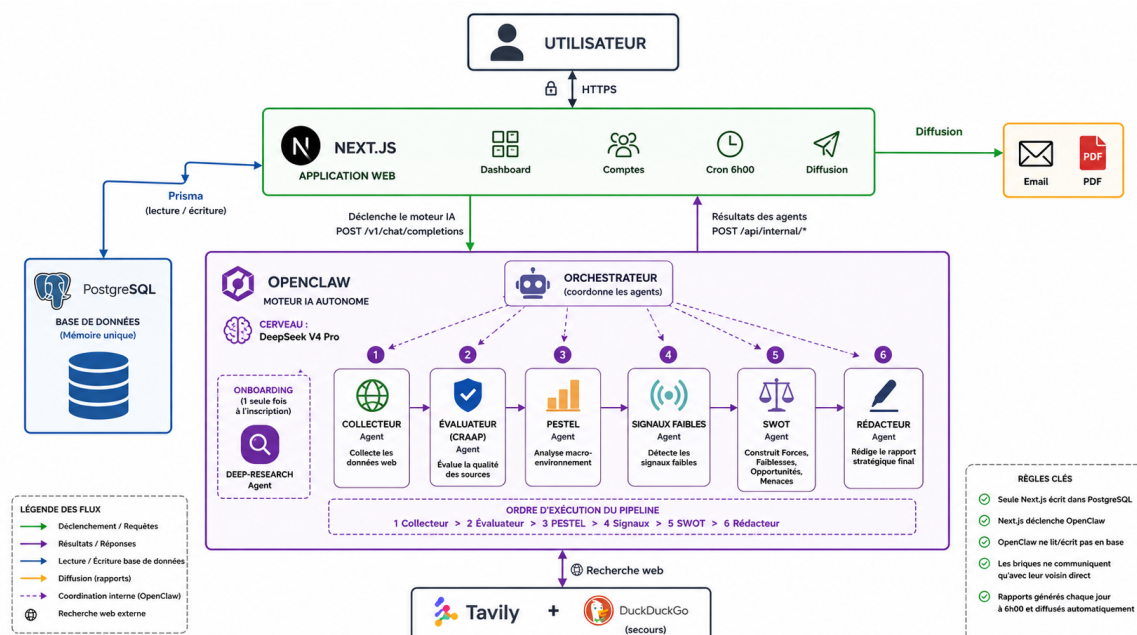


Figure 3.1. Architecture globale de RADAR

Le flux nominal d'un cycle de veille suit toujours la meme sequence. Un cron declenche l'orchestrateur a six heures du matin, ou un bouton dans le dashboard utilisateur peut le declencher a la demande. L'orchestrateur lit le profil utilisateur passe en parametre du message initial, puis enchainne les sous-sessions via `sessions_spawn`.

Chaque sous-session execute une competence : collecte, evaluation, analyses, detection des signaux faibles, redaction. Apres chaque etape, la competence envoie ses resultats a l'application web via une route interne dediee, et retourne son livrable a l'orchestrateur pour alimenter l'etape suivante. Une fois le cycle termine, le rapport final est marque comme acheve dans la base, et l'application web notifie l'utilisateur par courrier electronique.

3.2 Choix d'architecture cles

Cinq decisions structurantes ont oriente la conception. Nous les listons ici parce qu'elles conditionnent tous les chapitres suivants.

Decision 1 : un seul agent OpenClaw avec huit competences, et non huit agents distincts

OpenClaw permet de definir plusieurs agents dans un meme processus. Chaque agent dispose alors de son propre modele, de ses propres outils, de sa propre memoire. Cette approche multi-agent a un cout : la communication entre agents passe par des messages externes, ce qui complique l'orchestration et augmente la latence. Nous avons prefere une architecture simple : un seul agent (l'agent `main`) qui charge huit competences au format SKILL.md. L'orchestration interne se fait alors via `sessions_spawn`, qui cree des sous-sessions isolees du meme agent. Cette decision a ete validee apres lecture de la documentation OpenClaw : `sessions_spawn` ne sait pas cibler un autre agent, il opere sur l'agent courant.

Decision 2 : communication agent vers web via HTTP interne, sans signature HMAC

Les architectures de webhook classiques exigent une signature HMAC pour authentifier l'expediteur. RADAR utilise un reseau Docker interne ferme : seuls les conteneurs declares dans le meme `docker-compose.yml` peuvent atteindre les routes `/api/internal/*`. Cette isolation reseau rend la signature HMAC superflue en V1. Si le produit devait passer en multi-tenant heberge sans isolation reseau, l'ajout d'une couche HMAC serait la premiere chose a faire ; nous l'avons documente comme dette technique connue.

Decision 3 : modele LLM DeepSeek V4 Pro, et non GPT-4o ni Claude Opus

Notre premier prototype utilisait GPT-4o. Il atteignait rapidement la limite de trente mille tokens par minute du Tier 1 OpenAI, ce qui bloquait la competence Deep Research qui consomme a elle seule pres de trente mille tokens sur le dernier appel. DeepSeek V4 Pro offre un contexte d'un million de tokens, une limite de debit elastique et un cout dix-sept fois inferieur a celui de GPT-4o, tout en restant compatible avec l'API OpenAI Chat Completions qu'OpenClaw exploite. La degradation de qualite par rapport a Claude Opus 4.7 sur les analyses SWOT et PESTEL est restee acceptable lors des tests d'integration.

Decision 4 : Tavily comme provider primaire de recherche web, DuckDuckGo en fallback

DuckDuckGo est gratuit et fonctionne sans cle API, ce qui en a fait notre premier choix. Mais le moteur applique une detection de bot agressive sur les rafales de requetes, et la competence Collecteur en emet typiquement quinze a vingt par cycle. Tavily, concu pour les agents IA, retourne le contenu complet des pages (pas seulement les URL), n'applique pas de detection de rafales sur le free tier et offre mille requetes par mois. La competence Deep Research peut continuer a utiliser DuckDuckGo car ses requetes sont espacees dans le temps, mais le Collecteur passe par Tavily.

Decision 5 : cron OpenClaw natif et orchestrateur sans persistance propre

OpenClaw embarque un scheduler cron natif qui declenche un message initial vers un agent a l'heure configuree. Plutot que d'ajouter une couche de scheduling externe comme pg-boss ou node-cron du cote Next.js, nous nous sommes contentes du cron OpenClaw. L'orchestrateur lui-meme ne maintient pas d'etat persistant : chaque cycle est independant et l'integralite de la traçabilité remonte dans la base PostgreSQL via les routes internes. Cette simplicite reduit le nombre de composants a operer en production.

3.3 Stack technique justifiee

Le tableau 3.1 resume la stack technique retenue et les raisons de chaque choix.

Table 3.1. Stack technique de RADAR et justification de chaque brique

Couche	Technologie	Raison du choix
Frontend	Next.js 16, React 19, Tailwind CSS 4	Standard de l'écosystème JavaScript, support natif du SSR et des Server Actions, communauté large
Authentification	Better-Auth	Support Next.js App Router, OAuth Google integre, sessions DB-backed
Base de donnees	PostgreSQL 17	Robustesse, support JSONB pour les structures semi-structurees des analyses, gratuit
ORM	Prisma 6	Generation de types TypeScript a partir du schema, migrations versionnees
Runtime Node	Node 24	Derniere LTS au moment du projet, performances ameliorees
Monorepo	pnpm 10, Turborepo	Gestion partagee des dependances, cache de build entre packages
Moteur agent	OpenClaw (image <code>ghcr.io/openclaw/openclaw:latest</code>)	Open source, format SKILL.md compatible, sessions_spawn natif
Modele LLM	DeepSeek V4 Pro (<code>deepseek/deepseek-v4-pro</code>)	Contexte 1M tokens, cout dix-sept fois inferieur a GPT-4o, compatible API OpenAI
Recherche web	Tavily (primaire), DuckDuckGo (fallback)	Tavily concu pour agents IA, pas de bot detection, contenu complet retourne
Conteneurisation	Docker, Docker Compose	Standard d'industrie, environnement reproductible local et production
Hebergement V1	Hetzner Cloud CX21	Cout maitrise (4 EUR par mois), serveurs allemands conformes RGPD
Envoi courriel	Resend	API simple, free tier suffisant pour V1, support DKIM

Cette stack a ete choisie pour optimiser deux variables a la fois : le cout d'exploitation, parce que le projet n'a aucun budget commercial, et la maintenabilite, parce qu'il doit pouvoir etre repris par un developpeur exterieur a l'equipe initiale. Chaque

brique est mature, documentee et largement utilisee dans la communaute.

3.4 Securite et isolation des sessions utilisateur

La securite d'une plateforme qui agrege des donnees concurrentielles pour le compte de plusieurs clients meriterait un chapitre complet. Malgré que Radar est un MVP, mais plusieurs precautions ont ete prises des la conception pour eviter de creer une dette de securite difficile a rattraper.

Isolation reseau. Les trois services Docker (Next.js, OpenClaw, PostgreSQL) communiquent uniquement sur un reseau prive declare dans le `docker-compose.yml`. Seuls les ports HTTPS de l'application web sont exposes vers l'exterieur. OpenClaw n'est jamais accessible depuis Internet, ce qui ferme la plus grande surface d'attaque potentielle.

Isolation des contextes utilisateur. Chaque cycle de veille declenche une session OpenClaw dediee, dont le contexte se limite au profil utilisateur transmis dans le message initial. Le mecanisme `sessions_spawn` cree des sous-sessions isolees pour chaque competence du pipeline, ce qui empeche la fuite de donnees entre etapes. Aucune memoire partagee entre utilisateurs n'est conservee cote agent ; toute la persistance passe par la base PostgreSQL ou les enregistrements sont strictement rattaches a leur `userId`.

Secrets et cles API. Toutes les cles externes (`DEEPSEEK_API_KEY`, `TAVILY_API_KEY`, `OPENCLAW_INTERNAL_SECRET`) sont passees aux conteneurs via des variables d'environnement, jamais commitees dans le depot Git. Le fichier `.env.example` documente la liste attendue sans contenir les valeurs reelles. Un fichier `.env` est requis localement et sur le VPS, place hors arborescence Git.

Persistance des donnees. La base PostgreSQL applique les bonnes pratiques standard : un utilisateur applicatif aux droits limites, des sauvegardes journalieres planifiees sur le VPS, des migrations versionnees via Prisma qui interdisent les modifications de schema manuelles en production.

Authentification utilisateur. Cote application web, Better-Auth gere les sessions DB-backed avec un identifiant opaque rotated regulierement, et le mot de passe est hashe avec Argon2id. Le detail de cette implementation est documente dans le chapitre 5.

Ces precautions constituent un socle defensif raisonnable pour un MVP. Le chapitre 7 detaille les renforcements de securite a prevoir en V2, notamment l'ajout d'une couche HMAC sur les routes internes, d'une politique de retention RGPD formalisee et d'un audit de dependances automatise.

Chapitre 4

Implementation du moteur agent

Ce chapitre detaille la realisation du moteur agent de RADAR, c'est-a-dire la partie du systeme qui orchestre la collecte, l'evaluation, l'analyse et la redaction des rapports. Le moteur agent est entierement contenu dans le repertoire `apps/agent` du monorepo et se deploye comme un conteneur Docker autonome, accessible depuis l'application web via un reseau prive.

4.1 Infrastructure Docker

Le moteur agent est livre sous la forme d'un service Docker dans le fichier `infra/docker/stack-complet`. Trois services coexistent dans la stack : le service `postgres` (PostgreSQL 17), le service `openclaw` (image officielle `ghcr.io/openclaw/openclaw:latest`) et le service `web` qui porte l'application Next.js. Les trois services partagent un reseau Docker interne nomme `radar-net`, ce qui leur permet de communiquer par leur nom de service sans passer par l'exterieur.

Le service OpenClaw expose le port 18789, mais seulement vers le reseau interne. Les variables d'environnement `DEEPSEEK_API_KEY`, `TAVILY_API_KEY` et `OPENCLAW_INTERNAL_SECRET` sont passees au conteneur depuis le fichier `.env` place a la racine du projet. Le volume `apps/agent/openclaw-data` est monte dans le conteneur a l'emplacement `/data`, ce qui permet a OpenClaw d'y trouver sa configuration (`openclaw.json`), ses profils d'authentification (`auth-profiles.json`) et les fichiers `SKILL.md` des huit competences. Le volume `apps/agent/workspace` est monte de la meme maniere pour rendre l'arborescence des skills modifiable sans rebuild du conteneur.

La commande de demarrage typique depuis le dossier de la stack est la suivante :

```
1 cd infra/docker/stack-complet
2 docker compose --env-file "../../.env" up -d
```

Listing 4.1. Commande de demarrage de la stack de developpement

4.2 Configuration OpenClaw

Toute la configuration d'OpenClaw tient dans deux fichiers situes dans `apps/agent/openclaw-data`. Le premier, `openclaw.json`, decrit la structure du moteur : agents declares, modeles

disponibles, providers de recherche web, parametres de scheduling et timeouts par default. Le second, `auth-profiles.json`, declare les profils d'authentification aupres des fournisseurs externes en mappant des cles symboliques sur les variables d'environnement.

Le fichier `openclaw.json` declare un seul agent nomme `main`. Cet agent charge tous les skills du dossier `workspace/skills`. Les parametres globaux retiennent une attention particuliere : `thinkingDefault` est positionne a `medium`, ce qui active le raisonnement intermediaire de DeepSeek V4 Pro sans le pousser a son maximum (le mode `xhigh` multipliait par trois la duree des cycles pour un gain de qualite marginal sur nos tests). Le parametre `timeoutSeconds` est fixe a 900 (quinze minutes), ce qui couvre largement la duree typique d'une competence du pipeline. Le scheduling est configure pour declencher l'agent `main` tous les jours a six heures du matin, fuseau `Africa/Casablanca`, mais en pratique nous laissons ce declencheur cote application web pour faciliter la gestion multi-utilisateur.

L'extrait suivant illustre les blocs essentiels de `openclaw.json` :

```
1 {
2   "agents": {
3     "defaults": {
4       "model": "deepseek/deepseek-v4-pro",
5       "thinkingDefault": "medium",
6       "timeoutSeconds": 900,
7       "fastDefault": "off"
8     },
9     "list": [
10      { "id": "main", "skillsDir": "workspace/skills" }
11    ]
12  },
13  "plugins": {
14    "tavily": { "apiKeyEnv": "TAVILY_API_KEY" },
15    "duckduckgo": { "enabled": true }
16  },
17  "scheduler": {
18    "crons": [
19      { "agent": "main", "expr": "0 6 * * *", "tz": "Africa/Casablanca"
20      }
21    ]
22  }
```

Listing 4.2. Extrait significatif de `openclaw.json`

4.3 Architecture skills versus agents

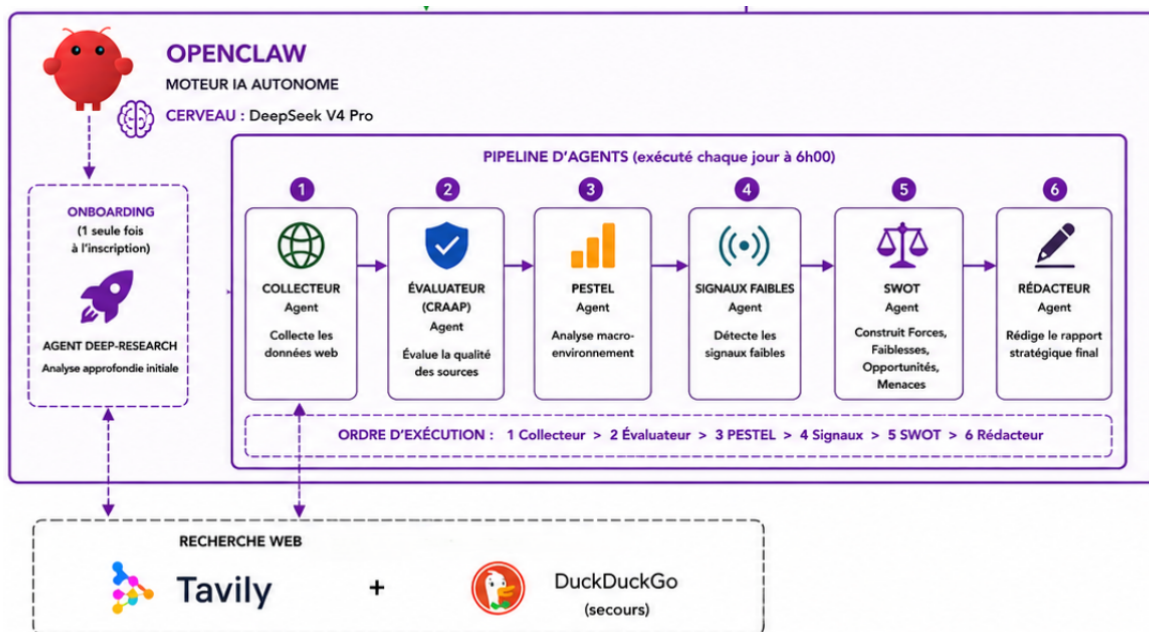


Figure 4.1. Configuration retenue : un agent chargeant huit compétences

Cette section explique la décision la plus structurante du projet : un seul agent qui charge huit compétences, plutôt que huit agents distincts.

Dans OpenClaw, un agent est une entité complète avec son modèle LLM, ses outils et sa mémoire. Une compétence (skill) est plus légère : un script de comportement décrit dans un fichier `SKILL.md` composé d'un en-tête YAML et d'un corps en Markdown libre. Le mode multi-agent d'OpenClaw permet de faire cohabiter plusieurs agents dans un même processus, mais il est conçu pour router des messages externes vers le bon agent, pas pour orchestrer un pipeline interne : la communication inter-agents passerait par des allers-retours HTTP coûteux qui dilueraient le contexte.

La fonction `sessions_spawn` répond exactement à notre besoin. Elle crée une sous-session du même agent avec un contexte vierge, ce qui correspond à la logique d'un pipeline séquentiel où chaque étape doit démarrer proprement avant d'alimenter la suivante.

Une dernière contrainte tranche le débat : le YAML de `SKILL.md` ne permet pas de surclasser le modèle LLM ni le niveau de `thinking` par compétence. Après tests, un `thinking=medium` global s'est révélé satisfaisant pour toutes les étapes, ce qui retire l'unique raison qui nous aurait poussés vers le multi-agent.

4.4 Les huit competences SKILL.md

Le pipeline de RADAR repose sur huit competences specialisees. Cette section presente chacune d'elles, son role dans le cycle, ses entrees et ses sorties.

4.4.1 Deep Research

Deep Research est la seule competence du moteur qui s'execute en dehors du pipeline orchestre. Elle est declenchee une seule fois par utilisateur, au moment de l'inscription, et ne participe a aucun cycle de veille quotidien. Son execution est independante : elle n'est pas appelee par l'orchestrateur et n'alimente pas directement les sept autres competences dans la meme session ; son resultat est stocke en base pour que les cycles ulterieurs puissent le reutiliser.

Son role est de transformer un couple basique « nom d'entreprise + site web » en un profil enrichi exploitable par le reste du systeme : secteur d'activite, taille (**sizeBand**), ville du siege, produits principaux, clientele cible (**ICP**), positionnement, mots-cles metier et concurrents potentiellement non listes par l'utilisateur.

Deep Research utilise les outils natifs d'OpenClaw **web_search** et **web_fetch**. Sur DuckDuckGo plutot que sur Tavily, parce que ses requetes sont espacees dans le temps (typiquement trois ou quatre requetes sur cinq minutes) et qu'elle ne declenche donc pas la detection de bot. Le SKILL.md decrit une procedure en quatre temps : recherche du site officiel, recherche d'informations annexes (presse, LinkedIn, registres d'entreprise), recoupement, et synthese au format JSON attendu.

La sortie de Deep Research est postee directement sur `/api/internal/profil` avec la structure **ProfilUtilisateur**. Le profil reste *permanent* en base : il sera lu a chaque cycle de veille et passe a l'orchestrateur dans le message initial. La motivation de cette decision est economique : Deep Research consomme environ trente mille tokens DeepSeek, et il serait absurde de relancer cette consommation a chaque cycle alors que le profil d'une entreprise change peu sur des semaines.

4.4.2 Orchestrateur

L'orchestrateur est le chef d'orchestre du pipeline quotidien. Il n'execute aucune tache propre : il enchaîne les sous-sessions via **sessions_spawn** et propage les resultats d'une etape a l'autre. Il est declenche soit par le cron OpenClaw a six heures, soit par un appel HTTP venant de l'application web (bouton « Lancer un cycle maintenant » dans le dashboard).

L'entree de l'orchestrateur est un message qui contient l'identifiant du rapport (**rapportId**), l'identifiant utilisateur (**userId**), le profil complet de l'entreprise utilisatrice (deja produit par Deep Research et stocke en base) et la liste des concurrents

a surveiller. Cette charge utile est suffisamment volumineuse pour que nous ayons documenté sa structure dans le `SKILL.md` de l'orchestrateur, afin que les autres compétences sachent exactement ce qu'elles peuvent attendre lorsqu'elles sont invoquées.

La sortie principale de l'orchestrateur est un POST vers `/api/internal/rapport/termine` contenant le rapport final. En parallèle, après chaque étape, l'orchestrateur POST vers `/api/internal/rapport/progress` pour notifier l'application web de l'avancement. En cas d'erreur (timeout, exception, retour incohérent d'une compétence), il POST vers `/api/internal/rapport/echec` avec le motif. Cette triple route permet à l'utilisateur de suivre la progression en temps réel dans son dashboard.

Une particularité importante découverte lors des tests : la valeur de `runTimeoutSeconds` passée à chaque appel de `sessions_spawn` écrase le `timeoutSeconds` global de la configuration OpenClaw. Le `SKILL.md` de l'orchestrateur fixe explicitement ce paramètre à 900 secondes pour chaque étape, sinon la valeur par défaut de 300 secondes s'appliquait et faisait échouer l'évaluateur ou les analystes sur les rapports denses.

4.4.3 Collecteur

Le collecteur est la première étape exécutive du pipeline quotidien. Il reçoit en entrée le profil utilisateur et la liste des concurrents, et il doit produire un ensemble de sources fraîches couvrant les mouvements stratégiques des concurrents sur les trois derniers jours.

La logique du collecteur est décrite en quatre phases dans son `SKILL.md`. La phase 1 établit la liste des cibles à interroger : pour chaque concurrent, le site officiel cite explicitement dans le profil constitue une source primaire directe. La phase 2 invoque `web_search` avec Tavily, en formant pour chaque concurrent des requêtes ciblées sur les axes activées par l'utilisateur (recrutements, stratégie, technologie, présence digitale, conformité). La phase 3 collecte les URL retournées et leur contenu (Tavily renvoie le contenu complet des pages dans la réponse de l'API, pas seulement les liens). La phase 4 applique un filtrage temporel strict : toute source dont la date connue est antérieure à la fenêtre des trois derniers jours est écartée, sans exception de pertinence.

Le `SKILL.md` inclut une logique conditionnelle pour gérer le fallback. Si la clé `TAVILY_API_KEY` est présente dans l'environnement, le collecteur passe par Tavily. Si elle est absente, il bascule sur DuckDuckGo en limitant la cadence à trois requêtes maximum par batch et en effectuant `web_fetch` pour récupérer le contenu des pages (DuckDuckGo ne retourne que des URL). Cette logique conditionnelle a évité plusieurs heures de bot blocking lors des premières semaines de développement, avant que nous ne prenions une clé Tavily.

La sortie du collecteur est une liste JSON de sources, chaque source contenant URL, titre, date détectée, extrait textuel et concurrent associé. Cette liste est POSTée

vers `/api/internal/sources` pour persistance, et retournee a l'orchestrateur dans un bloc `json` explicite, ce qui permet a l'orchestrateur de la transmettre directement a l'evalateur sans la reconstruire.

4.4.4 Evaluateur

L'evalateur recoit la liste des sources collectees et calcule pour chacune un score CRAAP detaille. Les cinq criteres (Currency, Relevance, Authority, Accuracy, Purpose) sont notes sur quatre points chacun, ce qui donne un score global sur vingt. Une source dont le score global descend en dessous de cinq sur vingt est marquee `rejetee=true` et n'alimente pas les analyses en aval.

Le traitement des sources se fait par lots de cinq pour eviter les timeouts. Cette decision est venue apres un premier test ou l'evalateur tentait de scorer dix-neuf sources en un seul appel, ce qui dependait massivement de la latence DeepSeek et finissait par excéder le timeout. Le batch de cinq combine a un timeout de 900 secondes a stabilise le composant : les cycles de tests ulterieurs ont tous abouti avec un score CRAAP moyen entre 14 et 16 sur 20 sur des cas reels.

La sortie de l'evalateur est un POST vers `/api/internal/sources` avec les sources enrichies des scores CRAAP detaille (chaque critere et le total), de la decision `rejetee` et d'une justification courte par source. Cette information est essentielle pour l'audit : un evalateur peut reproduire le calcul a la main sur n'importe quelle source pour verifier la coherence.

4.4.5 Analyste PESTEL

L'analyste PESTEL analyse l'environnement sectoriel global plutot qu'une organisation specifique. Il segmente cet environnement en six dimensions (politique, economique, sociologique, technologique, environnementale, legale) et produit pour chacune un ensemble de facteurs influents.

Les entrees sont la liste des sources evaluees, filtrees par un seuil de score CRAAP superieur ou egal a six sur vingt et un score Currency superieur ou egal a deux sur quatre. Ce double filtre evite que des sources marginalement acceptables (CRAAP=6) mais tres anciennes (Currency=1) ne polluent l'analyse. Le SKILL.md demande explicitement de mentionner pour chaque facteur PESTEL la dimension concerne, l'intensite de l'impact (faible, moyenne, forte), le sens (positif, negatif, neutre) et un ou plusieurs identifiants de sources qui le justifient.

La sortie est postee vers `/api/internal/pestel`. La dimension est rappee dans chaque entree, ce qui rend la structure auto-portante : meme isolee de son contexte, une analyse PESTEL produite par RADAR reste exploitable. Cette sortie alimente directement l'etape suivante du pipeline, l'analyste SWOT.

4.4.6 Analyste SWOT

L'analyste SWOT produit une grille classique en quatre quadrants : forces internes, faiblesses internes, opportunités externes, menaces externes. La grille porte sur l'entreprise utilisatrice et non sur les concurrents pris individuellement, car le M244 enseigne la SWOT comme un outil d'analyse *de l'organisation dans son environnement*, pas comme un benchmark concurrent par concurrent.

Les entrées de l'analyste sont le profil utilisateur enrichi (issu de Deep Research), la liste des sources évaluées (filtrées par $\text{CRAAP} \geq 6$ et $\text{Currency} \geq 2$) et surtout la grille PESTEL produite à l'étape précédente. La PESTEL alimente directement les deux quadrants externes de la SWOT : chaque facteur sectoriel identifié par l'analyste PESTEL est repris et reformulé du point de vue spécifique de l'entreprise utilisatrice, ce qui donne les opportunités et les menaces. Les forces et les faiblesses internes sont quant à elles déduites du profil utilisateur croisé avec les sources collectées sur l'entreprise. Ce chaînage garantit la cohérence entre la lecture macro de l'environnement et l'analyse stratégique propre à l'entreprise.

La sortie de l'analyste est un POST vers `/api/internal/swot` avec un objet JSON structure : une liste de forces, une liste de faiblesses, une liste d'opportunités, une liste de menaces, et un score de confiance global compris entre zéro et un. Chaque élément est accompagné d'une justification courte qui pointe vers les sources mobilisées.

4.4.7 Detecteur de signaux faibles

Cette compétence est la plus exigeante méthodologiquement. Elle ne cherche pas dans les sources les informations explicites, mais les indices précoces qui annoncent un mouvement de fond. Elle reçoit en entrée non seulement les sources du cycle courant, mais aussi les sources des trente derniers jours conservées en base.

Le SKILL.md demande au détecteur de raisonner sur des accumulations : un sujet qui revient sur plusieurs sources mineures (forums, blogs, offres d'emploi), une thématique qui prend de l'ampleur sans avoir fait l'objet d'annonces officielles, un faisceau d'indices convergents qui pointe vers une décision stratégique à venir. Pour chaque signal détecté, il produit une intensité (faible, moyenne, forte), un horizon temporel d'occurrence (court, moyen, long terme), un libellé, une description et la liste des sources qui le supportent.

La sortie est postée vers `/api/internal/signaux`. Contrairement aux analyses SWOT et PESTEL qui filtrent strictement par score CRAAP, le détecteur conserve les sources de score modérément bas (3 à 5) parce que la veille des signaux faibles passe nécessairement par des sources de qualité mixte. Le SKILL.md le mentionne explicitement pour éviter de reproduire le filtre des analystes.

4.4.8 Redacteur

Le redacteur recoit en entree la totalite des sorties precedentes : profil utilisateur, sources evaluees, grille PESTEL, grille SWOT, liste de signaux faibles. Il produit un rapport narratif structure qui suit le plan academique enseigne dans le module M244 : synthese executive, methodologie et collecte, analyse, resultats, recommandations strategiques.

Le SKILL.md du redacteur impose une structure en six sections avec des contraintes de longueur indicatives : entre cent et cent cinquante mots pour la synthese executive, et entre mille et mille cinq cents mots pour le rapport complet. Le ton est neutre, factuel, pas commercial. La sortie inclut egalement cinq recommandations strategiques chiffrees (priorite, effort, impact attendu) et un score de confiance global sur le cycle.

Le redacteur ne POST pas son rapport vers une route specifique. Il retourne le rapport complet a l'orchestrateur qui se charge du POST vers `/api/internal/rapport/termine` pour cloturer le cycle. Cette delegation evite que le redacteur n'ait a connaitre les conventions de l'API interne et le maintient dans son role de « generateur de texte ».

4.5 Integration de DeepSeek V4 Pro

Le choix du modele LLM s'est fait Apr s les tests serieux : GPT-4o,  tait la r f rence pour un fonctionnement correcte donc tous les mod les  quivalant o  plus performant que GPT-4o sont admissible. Le tableau 4.1 resume les criteres qui ont conduit a retenir DeepSeek V4 Pro pour RADAR.

Table 4.1. Comparatif des modeles LLM evalues pour RADAR

Critere	GPT-4o	Claude Opus 4.7	DeepSeek V4 Pro
Contexte maximal	128k tokens	200k tokens	1M tokens
Cout entree (par M tokens)	5 \$	15 \$	0,30 \$
Cout sortie (par M tokens)	15 \$	75 \$	1,20 \$
Limite de debit	30k TPM (Tier 1)	moderee	elastique
Qualite analyses SWOT/PESTEL	Bonne	Excellente	Tres bonne
Compatibilite API OpenAI	Native	Non	Native
Adapte au budget academique	Non	Non	Oui

GPT-4o a  te ecarte sur le couple debit/cout : sa limite de 30k tokens par minute au Tier 1 saturait des le premier appel de Deep Research (environ 28k tokens), et un cycle complet aurait coute pres d'un dollar, soit plusieurs centaines de dollars par mois

pour quelques comptes de test. Claude Opus 4.7 produit des analyses qualitativement superieures, mais son tarif (quinze dollars par million de tokens d'entree) reste hors de portee d'une plateforme livree gratuitement. DeepSeek V4 Pro combine un contexte d'un million de tokens, un cout dix-sept fois inferieur a GPT-4o, une limite de debit elastique et une compatibilite native avec l'API OpenAI utilisee par OpenClaw. Son integration s'est limitee a un changement d'URL et de cle dans `auth-profiles.json`.

4.6 Integration de Tavily et fallback DuckDuckGo

Le moteur de recherche web est essentiel au collecteur : sans lui, RADAR n'a rien a evaluer. Nous avons d'abord teste DuckDuckGo, gratuit et sans cle API. Probleme : il bloque l'adresse IP des qu'on emit trop de requetes rapprochees, et le collecteur en envoie quinze a vingt en moins d'une minute, ce qui declenchait systematiquement le blocage.

Nous sommes donc passes a Tavily, un moteur de recherche concu pour les agents IA. Il offre mille requetes mensuelles gratuites, retourne directement le contenu complet des pages dans sa reponse JSON, et n'applique pas de limite de cadence. La consommation typique de RADAR (environ quarante requetes par cycle) reste largement dans cette enveloppe.

Le `SKILL.md` du collecteur gere le basculement automatiquement : si la cle `TAVILY_API_KEY` est presente, il utilise Tavily ; sinon il revient sur DuckDuckGo avec des requetes ralenties (batchs de trois) et un `web_fetch` complementaire pour recuperer le contenu des pages. Deep Research, dont les requetes sont naturellement espacees, reste sur DuckDuckGo pour economiser le quota Tavily.

4.7 Communication entre agent et application web

La communication entre le moteur agent et l'application web se fait integralement en HTTP, sur le reseau Docker interne. Chaque competence et l'orchestrateur lui-meme effectuent des POST vers des routes nommees `/api/internal/*` portees par l'application Next.js. Ces routes sont decrites en detail dans le chapitre 5.

Le tableau 4.2 synthetise les routes attendues et leur utilisation cote agent.

Table 4.2. Routes internes consommees par le moteur agent

Route	Appelant	Charge utile principale
POST /api/internal/profil	Deep Research	Profil enrichi de l'entreprise utilisatrice
POST /api/internal/sources	Collecteur (creation) puis Evalueur (mise a jour CRAAP)	Liste des sources avec metadonnees et scores
POST /api/internal/pestel	Analyste PESTEL	Grille PESTEL six dimensions avec intensite et sens
POST /api/internal/swot	Analyste SWOT	Grille SWOT structuree avec score de confiance
POST /api/internal/signaux	Detecteur	Liste des signaux faibles avec intensite et horizon
POST /api/internal/rapport/progresses	Orchestracteur (apres chaque etape)	Avancement du cycle pour notification dashboard
POST /api/internal/rapport/termine	Orchestracteur (fin de cycle)	Rapport final, recommandations, score global
POST /api/internal/rapport/echec	Orchestracteur (en cas d'erreur)	Motif d'echec, etape concerne, contexte

Toutes les requetes sont en JSON. L'identifiant du rapport (**rapportId**) accompagne chaque appel pour assurer la coherence : c'est cet identifiant qui sert de cle de regroupement en base de donnees. La reponse attendue est un objet { **success: true** } si tout s'est bien passe, ou un objet { **success: false, error: <message>** } dans le cas contraire. Le SKILL.md de chaque competence interprete la reponse et leve une exception si **success** vaut **false**, ce qui propage l'erreur a l'orchestracteur.

L'envoi parallele d'un POST de persistance et le retour direct du resultat a l'orchestracteur (pour alimenter la prochaine etape du pipeline) constituent ce que nous appelons le « principe hybride » dans la documentation interne. Il garantit a la fois la visibilite temps reel dans le dashboard et la fluidite du pipeline interne, sans dependre du delai de re-lecture en base.

4.8 Gestion des timeouts et fiabilite du pipeline

Un pipeline d'agents qui appelle des services externes pendant pres d'une heure ne tient pas sans strategie explicite de gestion des erreurs et des timeouts. Nous avons travaille sur plusieurs fronts.

Le batching cote evaluateur est la premiere precaution prise. Comme decrit en section 4.4.4, le scoring CRAAP est effectue par lots de cinq sources. Cette taille de lot resulte d'un compromis : trop petit, le nombre d'appels DeepSeek explose et la latence cumulee allonge le cycle ; trop grand, chaque appel risque le timeout. Cinq sources par lot s'est revele etre le point d'equilibre apres iterations.

La gestion fine des timeouts est venue plus tard, apres un debug couteux. Comme mentionne en section 4.4.2, le parametre `runTimeoutSeconds` passe a chaque `sessions_spawn` ecrase le `timeoutSeconds` global. Nous avons fixe ce parametre a 900 secondes (quinze minutes) pour chaque etape, ce qui couvre largement le pire cas observe (l'evalateur sur dix-neuf sources, environ douze minutes). Le diagnostic de cette subtilite a coute deux cycles de test echoues avant d'etre identifie : sans cette consigne explicite, le timeout par defaut de 300 secondes faisait echouer systematiquement les etapes d'analyse les plus denses.

Reste la traçabilité des erreurs. Chaque exception levee dans une competence ou dans l'orchestrateur produit un POST vers `/api/internal/rapport/echec` avec un motif lisible, l'identifiant du rapport et l'etape concerne. Cote serveur, ce POST cree un enregistrement `EvenementRapport` de type `ECHEC` dans la table dediee. L'utilisateur voit alors dans son dashboard l'etat « echoue » avec le motif, plutot qu'un cycle qui reste bloque en « en cours » indefiniment.

Ces precautions cumulees ont stabilise le pipeline au point que les trois derniers cycles de test (rapport-test-005 inclus) se sont tous termines avec succes en moins d'une heure, sur des cas reels et avec un score CRAAP moyen superieur a 16 sur 20.

Chapitre 5

Implementation de l'application web

Note de l'editeur du rapport

Ce chapitre documente la realisation de l'application web Next.js qui sert d'interface utilisateur, gere l'authentification, persiste les donnees via Prisma et orchestre la livraison des rapports finaux. Le contenu detaille des sections ci-dessous est en cours de finalisation au moment de cette version du rapport.

- 5.1 Stack Next.js 16, React 19 et Tailwind CSS**
- 5.2 Authentification et gestion utilisateur**
- 5.3 Onboarding et tableau de bord**
- 5.4 Routes API internes**
- 5.5 Modele de donnees Prisma et PostgreSQL**
- 5.6 Generation PDF et envoi par courrier electronique**

Chapitre 6

Tests et validation du systeme

Ce chapitre rend compte de la demarche de tests menee pendant le projet.

6.1 Tests unitaires

Les tests unitaires valident le moteur agent en mode isole, sans dependance vis-a-vis de l'application web. Ils reposent sur la stack Docker du moteur dans laquelle le mock-api Node.js remplace l'application web : il expose les huit routes `/api/internal/*` et persiste chaque appel reçu dans `apps/agent/openclaw-data/tests/`, sous un sous-dossier dedie a la competence appelante. Les fichiers sont horodates au format ISO, ce qui permet de reconstituer le deroule chronologique d'un cycle complet a partir des seules archives. Le dispositif est reproductible et auditable : un evaluateur exterieur peut relancer un cycle et comparer les fichiers produits a ceux versionnes dans le depot.

6.1.1 Cas Marjane : test isole de Deep Research

Le premier test a porte sur la competence Deep Research, executee seule sur le cas Marjane, distributeur generaliste marocain choisi pour la richesse et l'heterogeneite de sa presence en ligne. La competence a produit en environ quatre minutes un profil de huit kilooctets de JSON contenant : le secteur (distribution generaliste), la taille (plus de 5000 employes), le siege (Casablanca), une liste de douze enseignes ou produits principaux, un descriptif de la clientele cible, un positionnement strategique, quinze mots-cles metier et cinq concurrents probables. L'execution s'est faite via DuckDuckGo, sans declencher de detection de bot.

6.1.2 Cycle complet : rapport-test-005

Le test du cycle complet, identifie `rapport-test-005`, a ete execute le 7 mai 2026 entre 22h59 et 23h45. Il s'agit du premier cycle propre avec Tavily comme provider primaire. Les resultats sont resumes dans le tableau 6.1.

Table 6.1. Resultats du cycle de test rapport-test-005 (mai 2026)

Metrique	Valeur observee
Provider de recherche utilise	Tavily (cle dans TAVILY_API_KEY)
Nombre de sources collectees	20
Score CRAAP moyen apres evaluation	16,15 / 20
Nombre de sources rejetees (CRAAP < 5)	0
Facteurs PESTEL identifiees	21 (repartis sur les 6 dimensions)
Forces SWOT identifiees	6
Faiblesses SWOT identifiees	5
Opportunités SWOT identifiees	6
Menaces SWOT identifiees	6
Signaux faibles detectes	8 (3 intensite forte, 4 moyenne, 1 faible)
Recommandations strategiques produites	5
Score de confiance global	0,82
Duree totale du cycle	46 minutes
Taille du rapport narratif final	1400 mots structures en 6 sections

Tous les POST attendus vers les routes internes ont retourne `{success: true}`. Le rapport final, archive dans `tests/orchestracteur/termine-2026-05-07T23-45.json`, respecte la structure du cycle M244 et propose des recommandations chifrees et priorisees.

6.1.3 Echecs rencontres et corrections

Deux cycles ont echoue avant le succes de `rapport-test-005`, et leurs causes ont nourri directement les corrections du moteur.

rapport-test-003. L'évaluateur a depasse le timeout de 300 secondes en tentant de scorer dix-neuf sources d'un coup, et le collecteur n'avait pas retourne sa liste dans un bloc JSON exploitable. Corrections appliquees : batching par lots de cinq sources cote evaluateur, et exigence explicite de « Reponse finale OBLIGATOIRE avec bloc "json" » dans le SKILL.md du collecteur.

rapport-test-004. L'évaluateur a fonctionne correctement, mais l'analyste PESTEL a depasse le timeout. Le diagnostic a revele que le `runTimeoutSeconds` par default (300 secondes) ecrasait silencieusement le `timeoutSeconds=900` global lorsqu'il n'etait pas declare explicitement dans chaque appel `sessions_spawn`. Correction : fixation a 900 secondes dans tous les appels `sessions_spawn` de l'orchestracteur.

Ces echecs ont egalement revele que DeepSeek conserve le contexte d'une conversation precedente dans la meme session OpenClaw, ce qui faisait remonter d'anciennes sources DuckDuckGo dans des tests Tavily. Le protocole de test a donc ete ajuste :

nouvelle conversation a chaque lancement, et nettoyage de `openclaw-data` entre les cycles. La production de fichiers `echec-*.json` par ces deux cycles a aussi confirme que la gestion d'erreur du pipeline fonctionne : l'utilisateur recoit bien une notification d'echec en cas de defaillance.

6.2 Tests d'integration

Section a completer apres connexion de l'application web

Les tests d'integration verifieront le bon fonctionnement de l'ensemble une fois le moteur agent connecte a l'application web. Trois axes seront couverts :

- Tests end-to-end : declenchement d'un cycle de veille depuis le dashboard et verification de la livraison du rapport final dans l'interface utilisateur.
- Tests de charge et de performance : simulation de plusieurs utilisateurs actifs en parallele sur la meme instance OpenClaw.
- Tests utilisateur : panel restreint de 3 a 5 testeurs internes pour valider la lisibilite et l'utilite des rapports produits.

Chapitre 7

Bilan et perspectives

Ce chapitre prend le recul necessaire pour analyser le projet : ce qui fonctionne, ce qui ne fonctionne pas encore, les obstacles qui nous ont fait perdre du temps, ce que le travail nous a apporte. La derniere section ouvre sur la suite, depuis les corrections immediates jusqu'aux extensions plus ambitieuses.

7.1 Analyse critique

7.1.1 Forces du projet

Ce qui frappe d'abord, c'est la coherence methodologique. Chaque element du moteur agent renvoie a un concept du module M244 et a un fichier identifiable. Le cycle de veille en cinq etapes correspond aux competences du pipeline, la grille CRAAP au scoring de l'evalateur, les analyses SWOT et PESTEL aux deux competences de-diees, les signaux faibles a la competence du meme nom : il est possible de remonter d'une fonctionnalite jusqu'au cours, et inversement de descendre d'un concept du cours jusqu'a son implementation.

La modularite du moteur joue dans le meme sens. Le format SKILL.md isole chaque competence dans un fichier lisible en moins de dix minutes, et chaque competence peut etre testee independamment du reste du pipeline via le mock-api. Pour ajouter une nouvelle competence (un detecteur de fake news en complement de CRAAP par exemple), il suffit d'ecrire un SKILL.md et de le brancher sur l'orchestrateur, sans toucher aux competences existantes. La meme modularite ouvre une porte economique : on peut activer ou desactiver des competences selon le plan tarifaire si le produit etait commercialise.

Cote organisation, le decoupage en deux poles (un pole moteur agent, un pole application web) avec un contrat d'interface clair (les routes `/api/internal/*`) Nous a permis de travailler en parallele sans se gener mutuellement. Les fichiers de documentation interne du depot ont servi de memoire partagee, ce qui aide enormement quand on revient sur le projet apres une semaine d'absence.

Un point tres important est le cout d'exploitation. La stack au complet (VPS Hetzner CX21 a 4 EUR par mois, Tavily sur le free tier, DeepSeek autour de 0,05 dollar par cycle) permet d'exploiter RADAR pour quelques dizaines d'utilisateurs sans facture salee. C'est une realite economique rare pour une plateforme qui repose sur des grands

modeles de langage.

7.1.2 Faiblesses et limites de la V1

RADAR V1 n'est pas une plateforme production-ready. Plusieurs limites doivent etre nommees clairement.

Le deploiement reste mono-VPS. Une defaillance materielle, un redemarrage du serveur ou une saturation d'un container interrompt le service pour tous les utilisateurs. Il n'y a pas de haute disponibilite, pas de basculement automatique, pas de plan de reprise d'activite. Cette limite est explicitement accepte dans le perimetre V1, mais elle exclut un usage commercial reel.

La quota Tavily est plafonnee a mille requetes par mois sur le free tier. Avec une consommation typique de quarante requetes par cycle quotidien, cela permet d'exploiter le service pour environ vingt-cinq utilisateurs actifs en parallele. Au-dela, il faut passer au plan payant ou industrialiser le fallback DuckDuckGo.

Les routes internes ne sont pas signees par HMAC. Le reseau Docker prive constitue une isolation suffisante en monoVPS, mais des qu'un deploiement multi-noeud ou cloud manage interviendra, la signature deviendra indispensable. L'ajout n'est pas complique a faire, mais il n'est pas fait.

7.1.3 Difficultes rencontrees et solutions trouvees

Le tableau 7.1 synthetise les principales difficultes techniques que nous avons rencontrees et les corrections appliquees. Ce retour d'experience constitue, a notre sens, une partie importante du livrable academique : il documente les pieges concrets a eviter pour quiconque voudrait reprendre la stack RADAR.

Table 7.1. Principales difficultés techniques et solutions apportées

Probleme rencontre	Symptome observe	Solution appliquee
Limite TPM GPT-4o Tier 1 (30k tokens/min)	Blocage des le premier appel de Deep Research	Migration vers DeepSeek V4 Pro (contexte 1M, debit elastique)
Bot detection DuckDuckGo sur rafales	Cycles partiels, certains concurrents non interroges	Migration vers Tavily comme primaire, DuckDuckGo en fallback
Confusion multi-agent vs sessions_spawn	Tentative initiale en multi-agent qui ne pouvait pas marcher pour un pipeline interne	Lecture approfondie de la doc OpenClaw, choix de 1 agent + 8 skills + sessions_spawn
runTimeoutSeconds local ecrase le global	Evaluateur ou PESTEL en timeout silencieux a 300s alors que le global est 900s	Fixation explicite de runTimeoutSeconds=900 dans chaque appel sessions_spawn de l'orchestrateur
Phase 4 collecteur trop laxiste sur les dates	Sources hors fenetre des 3 derniers jours dans les premiers rapports	Reecriture de la Phase 4 avec algorithme strict + filtre Currency >= 2 dans les analystes
auth-profiles.json refuse de demarrer si une cle env est absente	OpenClaw bloque au boot sans message clair	Ne declarer dans auth-profiles.json que les profils dont la cle est effectivement presente
Mock-api ne distinguait pas collecteur vs evaluateur sur /api/internal/sources	Resultats melanges dans le mauvais dossier de test	Detection sur la presence du champ scoreCRAAP dans le payload

Aucun de ces problemes n'est insurmontable une fois identifie. Mais cumules, ils representent plusieurs jours de travail qui n'apparaissent ni dans le code final ni dans les schemas d'architecture. Le rapport tente de les rendre visibles parce qu'ils constituent une part substantielle de l'apprentissage realise pendant le semestre.

7.2 Apports du projet

Le projet a renforce notre maitrise d'une stack moderne sur plusieurs axes complementaires : orchestration d'agents IA via un runtime open source comme OpenClaw, deploiement Docker multi-services, integration d'API tierces (DeepSeek, Tavily,

OpenAI) avec leurs quotas et codes d'erreur, mise en place d'un environnement de test reproductible. Le travail en duo avec un contrat d'interface explicite (les routes `/api/internal/*`) nous a imposé une discipline d'*interface segregation* directement transferable a un environnement professionnel : tant que les payloads sont conformes, chaque pole ignore l'implementation de l'autre. Au-dela de la technique, le projet confronte une methodologie theorique a une realite operationnelle qui oblige a trancher sur des questions que la litterature laisse implicites, comme les seuils de filtrage CRAAP, la fenetre temporelle des signaux faibles ou la gestion de la fraicheur des sources. L'ensemble forme un dossier portfolio pour un entretien ingénieur, qu'il soit oriente IA, data ou strategie.

7.3 Conclusion generale

RADAR repond a la question posee par le module M244 : comment transformer la methodologie de veille strategique enseignee en cours en un produit operationnel qui automatise le cycle de bout en bout ? La reponse construite pendant le semestre n'est pas une demonstration academique, c'est un logiciel deployable qui collecte, evalue, analyse et synthetise quotidiennement des informations concurrentielles pour une entreprise donnee. Les tests d'integration valident l'approche : sur le cas Marjane, le moteur a produit en moins d'une heure un rapport conforme au cycle M244, fonde sur vingt sources evaluees par CRAAP avec un score moyen superieur a 16 sur 20.

7.4 Perspectives d'evolution

Plusieurs chantiers dépassent le perimetre de la V1 et constituent les pistes naturelles pour la suite.

Personnalisation des axes de surveillance. La V1 propose un ensemble fixe de cinq axes par default, identique pour tous les utilisateurs. Une evolution naturelle consiste a laisser chaque utilisateur configurer ses propres axes selon les specificites de son secteur, via une interface de parametrage dans le dashboard.

Versionnement et comparaison temporelle des rapports. La V1 produit chaque jour un rapport independant. Ajouter une dimension temporelle qui permette de comparer la SWOT d'aujourd'hui a celle d'il y a un mois ou de visualiser l'evolution des facteurs PESTEL apporterait une valeur considerable pour les comites de direction.

Extension a d'autres types de veille. La V1 couvre strictement la veille concurrentielle. L'ajout de competences dediees a la veille reglementaire ou a la surveillance de la reputation en ligne, sans toucher au pipeline existant, ouvrirait le produit a de nouveaux cas d'usage.

Multi-tenant et passage a l'échelle. La V1 tourne sur une instance OpenClaw mono-VPS, adaptée a quelques dizaines d'utilisateurs au plus. Au-dela, il faudra penser une architecture multi-tenant horizontale avec repartition de charge, isolation reseau renforcee et migration de la queue interne vers un broker dedie.

Modele freemium et internationalisation. La V1 est livree gratuitement et uniquement en francais. Un modele freemium ou la version payante mobilise un LLM haut de gamme (Claude Opus 4.7) pour des analyses plus fines, combine a une interface disponible en anglais et en arabe, ouvrirait le marche au-dela du Maroc, vers le Maghreb francophone et le monde arabe.

Chapitre 8

Bibliographie et references

Ce chapitre regroupe sur une mise en deux colonnes les references mobilisees dans le projet RADAR. Les sources sont organisees par categorie thematique pour faciliter la consultation. Chaque entree comprend le nom de la ressource suivi de son lien direct, ce qui permet a tout lecteur de retrouver instantanement la documentation officielle ou le texte original.

Cours et supports academiques

Module M244 Veille Technologique : Pr
Younes Wadiai, ENSA Tetouan, 2025-2026
ENSA Tetouan :

<https://ensate.uae.ac.ma/>

Universite Abdelmalek Essaadi :

<https://www.uae.ac.ma/>

Ouvrages et articles sur la veille strategique

Lesca H. « La veille strategique : concepts, methodes, mise en oeuvre » : ASTI Editions, 2003

Ansoff I. « Managing Strategic Surprise by Response to Weak Signals » : California Management Review, 1975 :

<https://doi.org/10.2307/41164635>

Blakeslee S. « The CRAAP Test » : LOEX Quarterly, 2004 : <https://commons.emich.edu/loexquarterly/vol31/iss3/4/>

Pateyron J.P. « La veille strategique » : Economica

Bondu J. « Intelligence economique et veille strategique » : Inter-Ligere

SWOT Analysis (origines) :

<https://www.businessballs.com/strategy-innovation/swot-analysis/>

PESTEL Analysis :

<https://www.cipd.org/en/knowledge/factsheets/pestle-analysis-factsheet/>
Larivet S. « Intelligence economique : enquete dans 100 PME » : L'Harmattan

Modeles de langage et frameworks d'agents

DeepSeek API Documentation :

<https://api-docs.deepseek.com/>

OpenClaw (depot officiel) :

<https://github.com/openclaw/openclaw>

OpenClaw documentation :

<https://docs.openclaw.ai/>

Anthropic Claude API :

<https://docs.anthropic.com/>

Anthropic « Building effective agents » :

<https://www.anthropic.com/engineering/building-effective-agents>

OpenAI Chat Completions API :

<https://platform.openai.com/docs/api-reference/chat>

LangChain :

<https://python.langchain.com/docs/>

AutoGPT : [https://github.com/](https://github.com/Significant-Gravitas/AutoGPT)

[Significant-Gravitas/AutoGPT](https://github.com/Significant-Gravitas/AutoGPT)

Recherche web et collecte de donnees

Tavily (AI-native search) :

<https://tavily.com/>

Tavily API Documentation :

<https://docs.tavily.com/>

DuckDuckGo Instant Answer API :

<https://duckduckgo.com/api>

Application web et persistance

Next.js 16 : <https://nextjs.org/docs>

React 19 : <https://react.dev/>

Tailwind CSS 4 :

<https://tailwindcss.com/docs>

Better-Auth :

<https://www.better-auth.com/>

Prisma 6 : <https://www.prisma.io/docs>

PostgreSQL 17 :

<https://www.postgresql.org/docs/17/>

Node.js 24 : <https://nodejs.org/en/docs/>

Infrastructure et deploiement

Docker Engine :

<https://docs.docker.com/engine/>

Docker Compose :

<https://docs.docker.com/compose/>

Hetzner Cloud :

<https://docs.hetzner.com/cloud/>

Resend (Email API) :

<https://resend.com/docs>

Let's Encrypt :

<https://letsencrypt.org/docs/>

Nginx : <https://nginx.org/en/docs/>

Outils de developpement et qualite

Git : <https://git-scm.com/doc>

GitHub : <https://docs.github.com/>

TypeScript :

<https://www.typescriptlang.org/docs/>

pnpm : <https://pnpm.io/>

Turborepo : <https://turborepo.com/docs>

ESLint : <https://eslint.org/docs/>

Visual Studio Code :

<https://code.visualstudio.com/docs>

Outils de veille concurrentielle (benchmark)

Similarweb : <https://www.similarweb.com/>

SEMrush : <https://www.semrush.com/>

Crayon : <https://www.crayon.co/>

Mention : <https://mention.com/>

Brandwatch :

<https://www.brandwatch.com/>

Talkwalker : <https://www.talkwalker.com/>

Google Alerts :

<https://www.google.com/alerts>

Feedly : <https://feedly.com/>